HEXORA

SPECIFICATION & PROTOTYPE

Video games, digital and physical — P2 · Week 7

Motroc Cosmin Alexandru
Catalog & admin lead

Nica Rares-Octavian
Payments & security lead

Iordachescu Iulian Daniel
Accounts & shopping lead

E-Commerce · 2026–2027
P2 · Hexora

STATUS SINCE P1

From a plan to a running store

The data model, component architecture and API surface are frozen. The three riskiest areas — catalog, accounts and payments — run against a seeded MySQL database.

13
tables in the frozen Prisma schema

32
purchasable editions from 16 games and 4 categories

9
test files, one command: `npm test`

3
branches, each with its own author and review

Frozen at P2

ERD, endpoint list, wireframes, auth contract, payment service contract. Later changes need a written request.

Working today

A1 catalog, A2 search, A4 accounts, A10 groundwork, A6 webhook, admin/local MySQL setup, seed data.

Next: weeks 8–12

A3 cart, A5 checkout, A7 stock UI, A8 history, A9 admin, A11 responsive, A12 full test suite, B4 + B5.

REQUIREMENTS A1–A4

Catalog, search, accounts

A1

Product catalog

Server-rendered paginated catalog (24/page), category pages and a product page with gallery, price, stock, platform, format and sibling edition.

• IMPLEMENTED

A2

Basic search

Case-insensitive search on product names with filters preserved in the URL and an explicit “No games found” state.

• IMPLEMENTED

A3

Shopping cart

One cart per authenticated user; add, update and remove with server-recalculated totals and stock caps. Persists after logout.

• WEEKS 8–9

A4

User accounts

Register, login, logout, bcrypt hashes and a signed httpOnly JWT. Address CRUD with exactly one default address.

• IMPLEMENTED

What the demo proves

4 categories · 16 games · 32 editions · 64 generated images, all read from MySQL at 127.0.0.1:3307. A missing cover falls back gracefully and a zero-stock edition is visibly out of stock.

Server-side by design

Prisma is imported only in `src/server/*` and route handlers. Passwords never leave the server as hashes; the session cookie is httpOnly, SameSite=Lax and signed with HS256.

Catalog src/app/(shop)/catalog · categories/[slug] · products/[slug] **Queries** src/server/queries/catalog.ts **Seed** prisma/seed-products.ts **Accounts** src/lib/auth.ts · src/app/account

REQUIREMENTS A5–A8

Checkout, payment, stock and history

A5

Checkout

Address → delivery method (digital 0 lei, courier 19,99 lei) → order summary → card payment. The confirmation page shows the order number.

- WEEKS 8–9

A6

Sandbox payment

Only a signed `payment_intent.succeeded` webhook marks an order Paid. A browser redirect never changes status; a failure stays Unpaid.

- WEBHOOK NOW

A7

Stock management

A conditional `stock >= quantity` decrement inside the paid transaction; rollback on conflict. Cancellation restores stock exactly once.

- SERVICE NOW

A8

Order history

Customer order list and detail with date, items, total, payment state and status history.

- WEEK 9

1 · Checkout
Server reloads the cart, validates stock, creates Order NEW / UNPAID.

2 · PaymentIntent
Amount computed server-side; Stripe.js confirms the test card.

3 · Webhook
Signature verified, event stored once, order → PAID, stock −qty, history appended.

4 · Confirmation
Page reads the database; the browser redirect is never trusted.

Webhook src/app/api/webhooks/stripe/route.ts **Service** src/server/services/payments.ts **Docs** docs/payments.md · scripts/test-webhook.mjs

REQUIREMENTS A9–A12

Admin, security, responsive, tests

A9

Administration panel

Separate ADMIN role, product/category CRUD with image upload, order list filtered by status and audited transitions New → Paid → Shipped → Delivered.

- WEEKS 10–11

A10

Basic security

Zod validation, same-origin check on mutating routes, security headers, server-side role guards and Prisma parameterisation.

- GROUNDWORK NOW

A11

Responsive design

All customer pages usable at 360 px: 2-column product grid on mobile, 4-column desktop, no horizontal scroll.

- IMPLEMENTED

A12

Automated tests

Vitest, one command. Payment confirmation, stock, auth, sessions, catalog and money covered; cart and totals join with A3.

- GROWING

Why the split matters

A10 is not a UI concern: every route calls `requireUser` / `requireRole` and every mutating request passes `assertSameOrigin` before touching data.

What P3 will grade

Only working behaviour. The prototype already exercises the parts that are easy to fake: database reads, password hashing, cookie sessions, webhook signatures and stock transactions.

Security src/lib/security.ts · next.config.mjs **Tests** 9 files · vitest · npm test **Responsive** docs/screenshots/*-360.png

BONUSES

B4 sales dashboard · B5 discount codes

B4 · 3 points

Daily sales, top products, average order

Admin dashboard over a selected interval: a daily sales chart, the top five products by revenue and the average order value. Computed with grouped SQL aggregates, not hard-coded numbers.

- **Data:** Order.paidAt, Order.total, OrderItem.quantity.
- **Filter:** from/to dates, default last 30 days.
- **Owner:** Cosmin · week 11.

B5 · 3 points

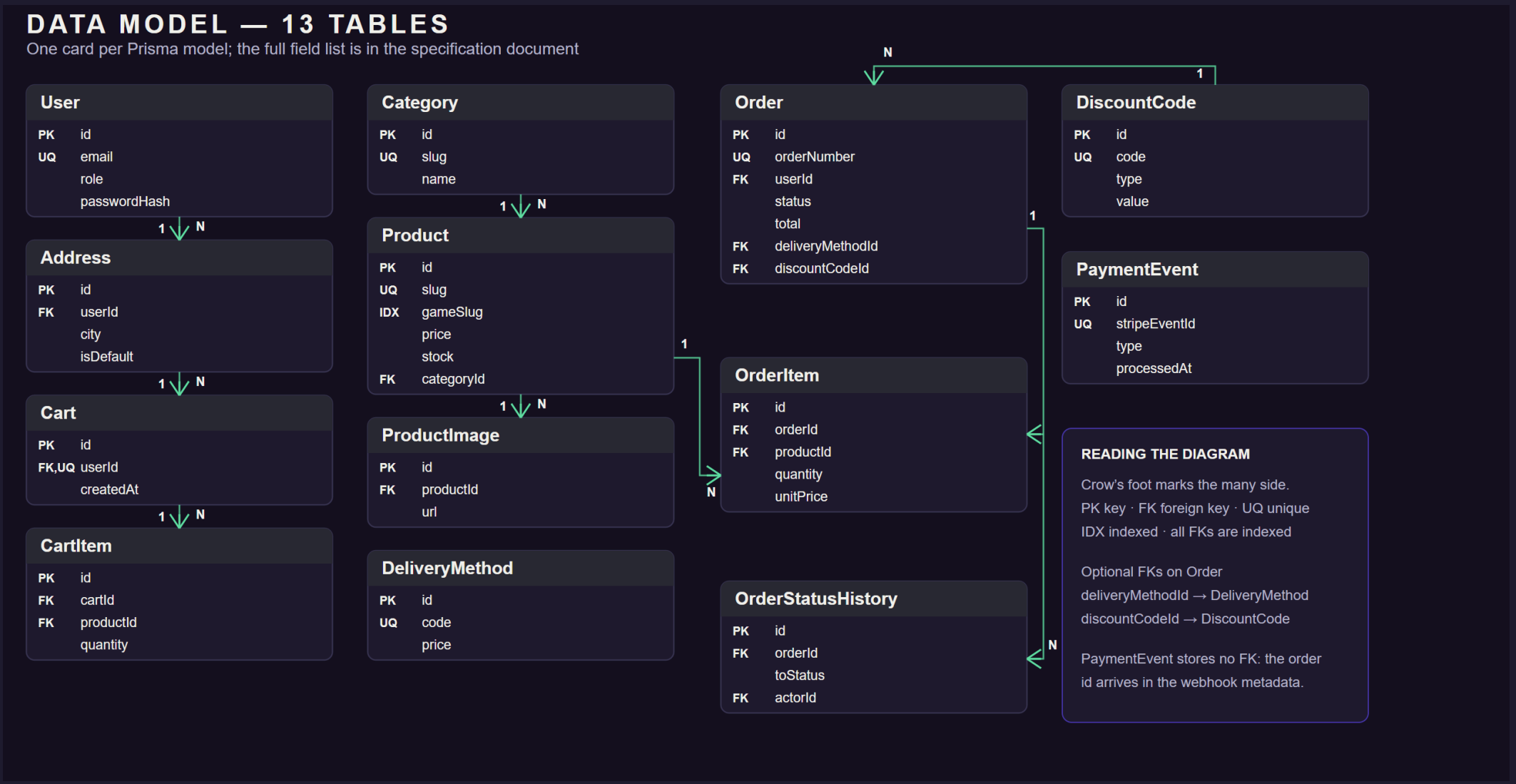
Percentage or fixed discounts

A code has a type, value, optional expiry and maximum number of uses. Validation happens on the server at checkout; the used counter increments once per paid order.

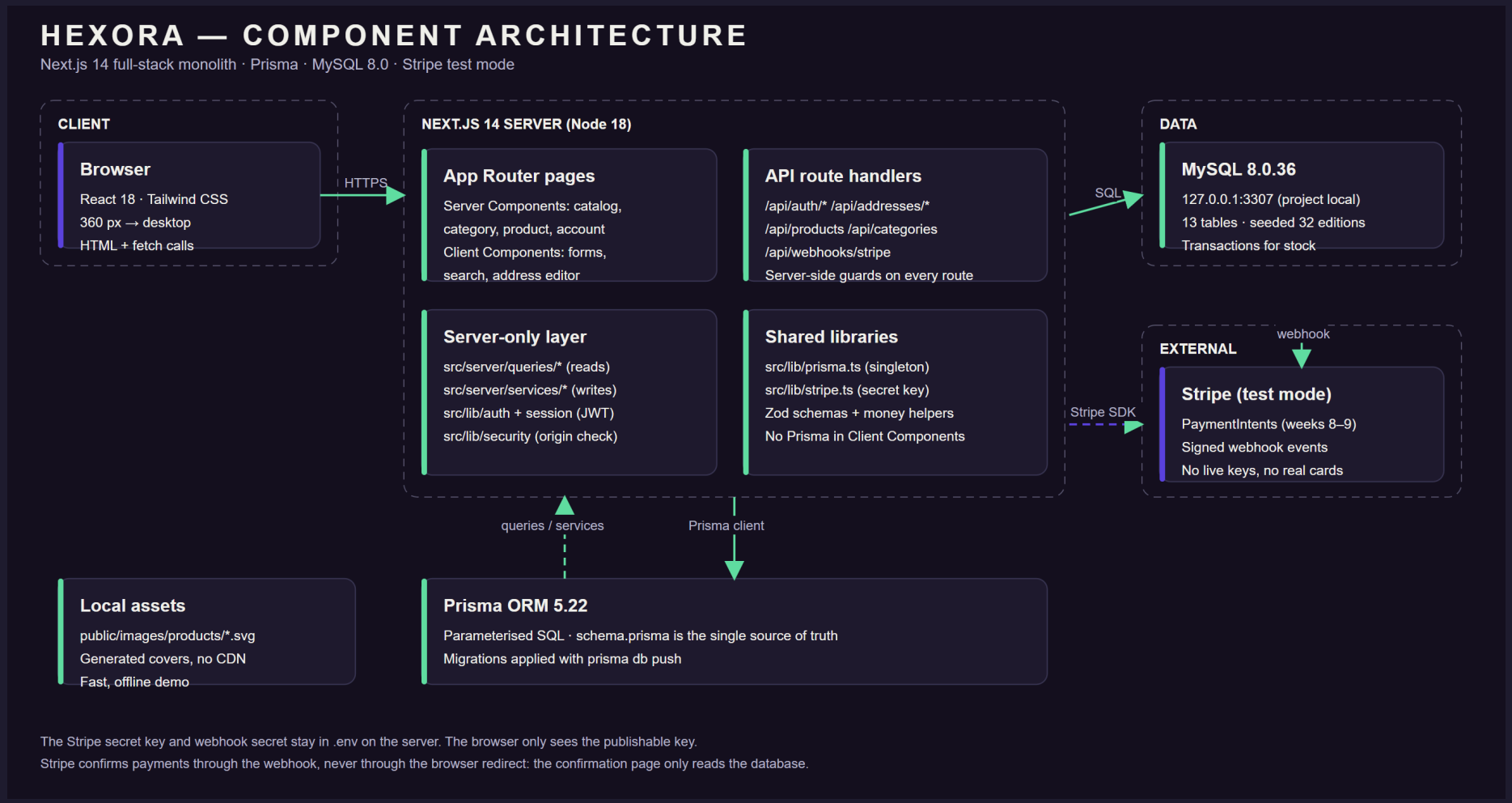
- **Model:** DiscountCode + Order.discountCodeId.
- **Admin:** list, create, edit, deactivate.
- **Owner:** Iulian · week 11.

Only B4 and B5 are claimed. Bonuses score only after A1–A12 work, so both land in week 11, after checkout and admin.

13 tables, one edition per product



Next.js full stack, Prisma, MySQL, Stripe



API SURFACE

Implemented at P2

METHOD	ROUTE	ACCESS	PURPOSE
GET	/api/products	public	Paginated catalog, max 24/page; q, category, page, filters
GET	/api/products/[slug]	public	Product detail with images and sibling editions
GET	/api/categories	public	Categories with active edition counts
POST	/api/auth/register	public	Create account, hash password, start session
POST	/api/auth/login	public	Verify password, set signed session cookie
POST	/api/auth/logout	authenticated	Clear the session cookie
GET	/api/auth/me	authenticated	Current user
GET	/api/addresses	customer	List own addresses, default first
POST	/api/addresses	customer	Create address, first one becomes default
PATCH	/api/addresses/[id]	owner	Update address or promote to default
DELETE	/api/addresses/[id]	owner	Delete address, promote the next one
POST	/api/webhooks/stripe	Stripe signature	Verify, store once, mark order paid, decrement stock

Access levels are enforced on the server; hiding a link in the UI is never the control.

API SURFACE

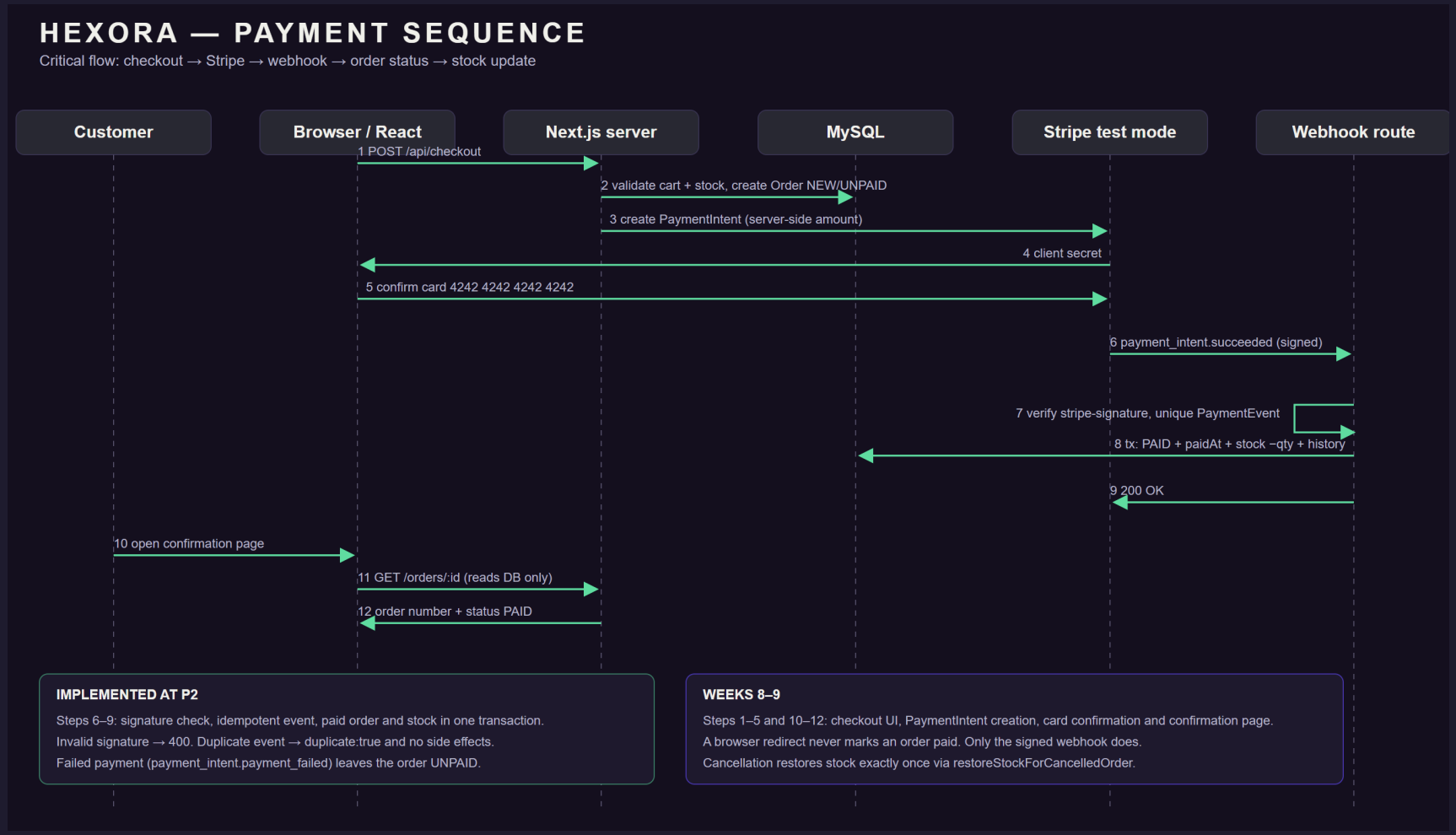
Frozen for P3

METHOD	ROUTE	ACCESS
GET	/api/cart	customer
POST	/api/cart/items	customer
PATCH	/api/cart/items/[id]	owner
DELETE	/api/cart/items/[id]	owner
POST	/api/checkout	customer
GET	/api/orders	customer
GET	/api/orders/[id]	owner
GET	/api/delivery-methods	public
POST	/api/discounts/validate	customer

METHOD	ADMIN ROUTE	ACCESS
GET/POST	/api/admin/products	ADMIN
PATCH/DELETE	/api/admin/products/[id]	ADMIN
POST	/api/admin/products/[id]/images	ADMIN
GET/POST	/api/admin/categories	ADMIN
PATCH/DELETE	/api/admin/categories/[id]	ADMIN
GET	/api/admin/orders	ADMIN
PATCH	/api/admin/orders/[id]/status	ADMIN
GET	/api/admin/dashboard	ADMIN (B4)
GET/POST/PATCH	/api/admin/discounts	ADMIN (B5)

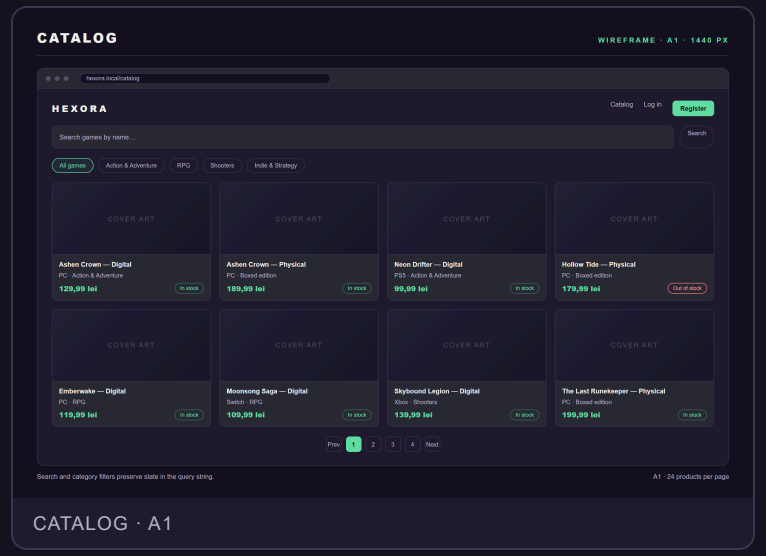
CRITICAL FLOW

Payment sequence

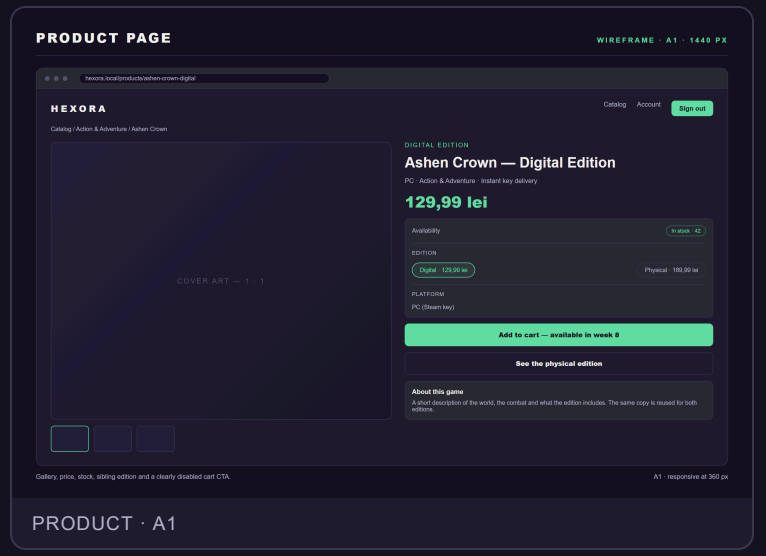


USER INTERFACE

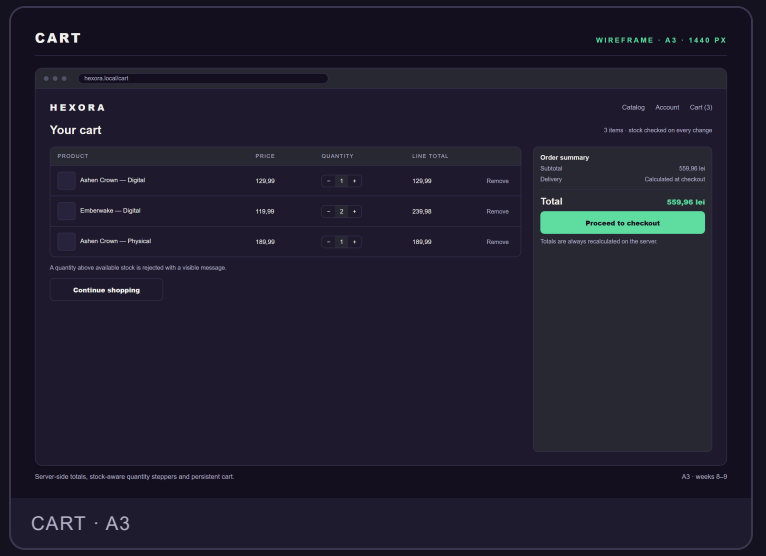
Wireframes — catalog, product, cart



CATALOG · A1



PRODUCT · A1

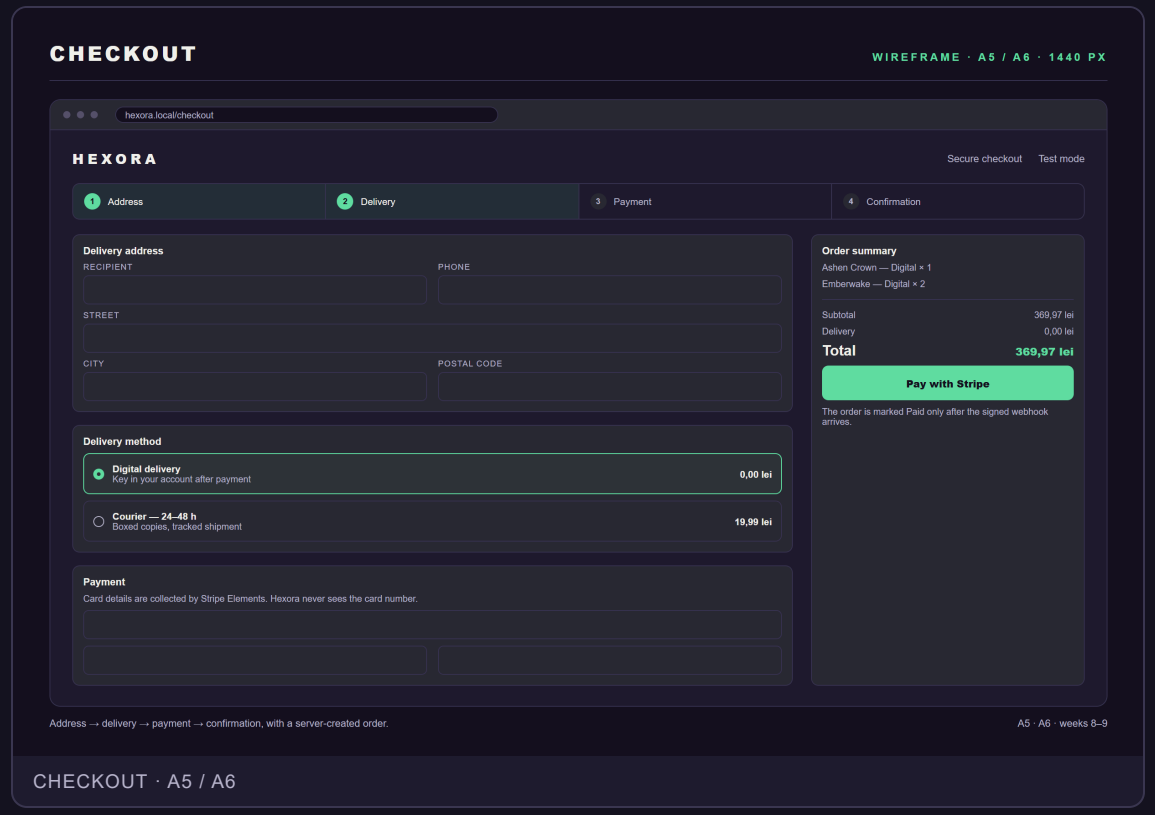


CART · A3

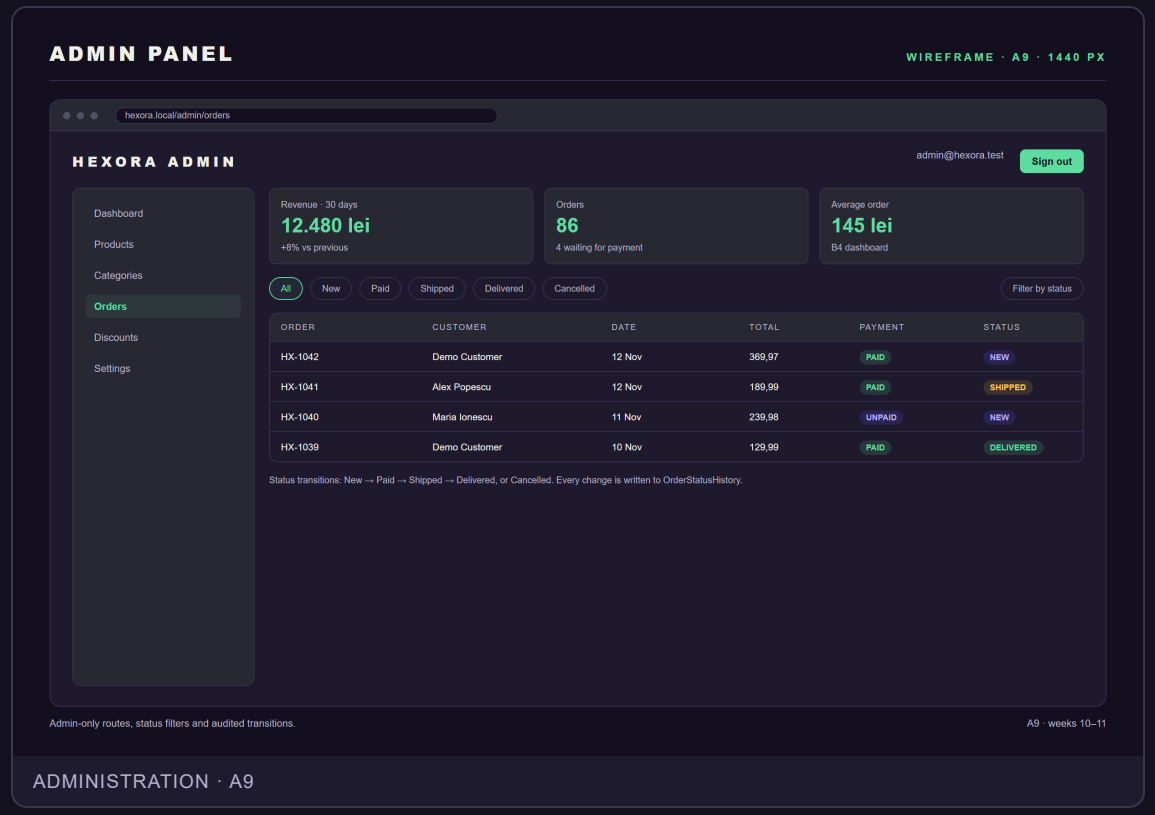
Wide screens shown; every customer page keeps the same hierarchy at 360 px, with the product grid collapsing from four columns to two.

USER INTERFACE

Wireframes — checkout, administration



CHECKOUT · A5 / A6

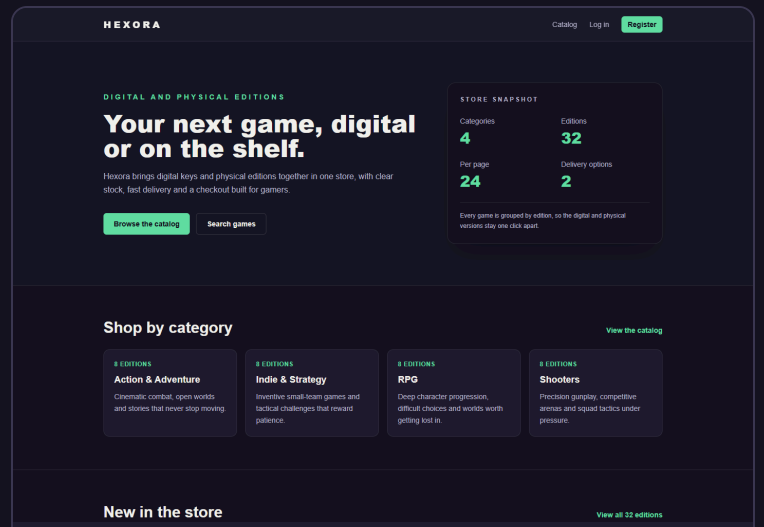


ADMINISTRATION · A9

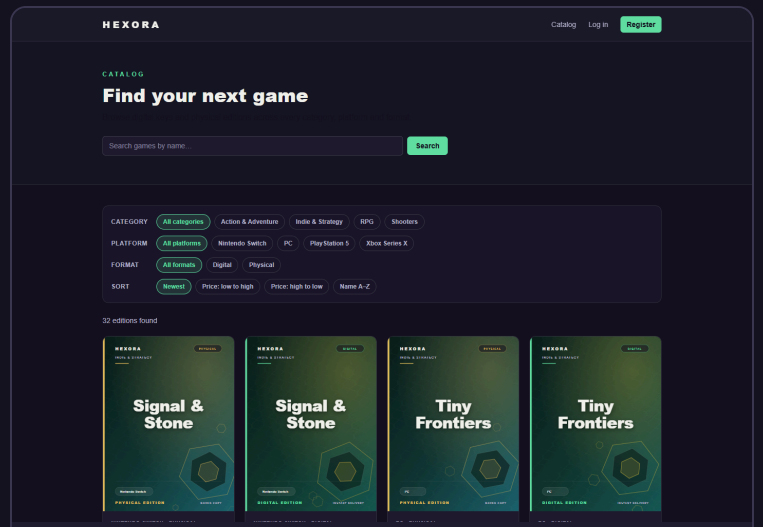
Checkout makes the delivery cost and the payment state explicit. The admin panel exposes role-guarded CRUD, status filters and the audited transition path.

WORKING PROTOTYPE

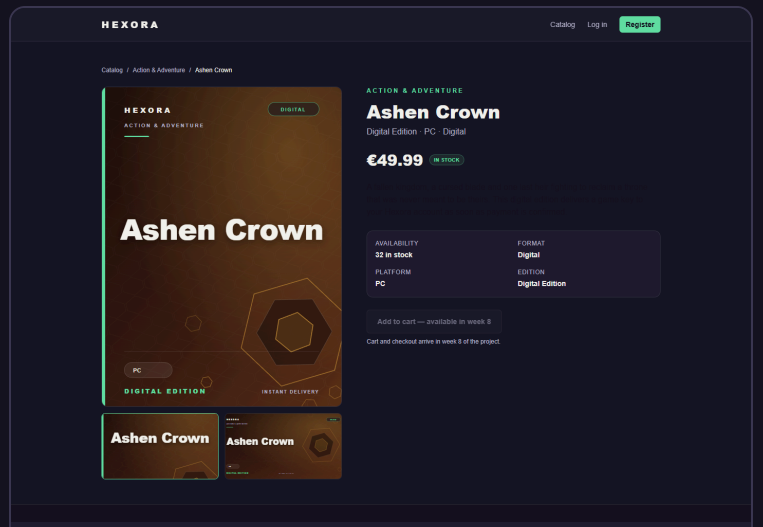
Live data from MySQL



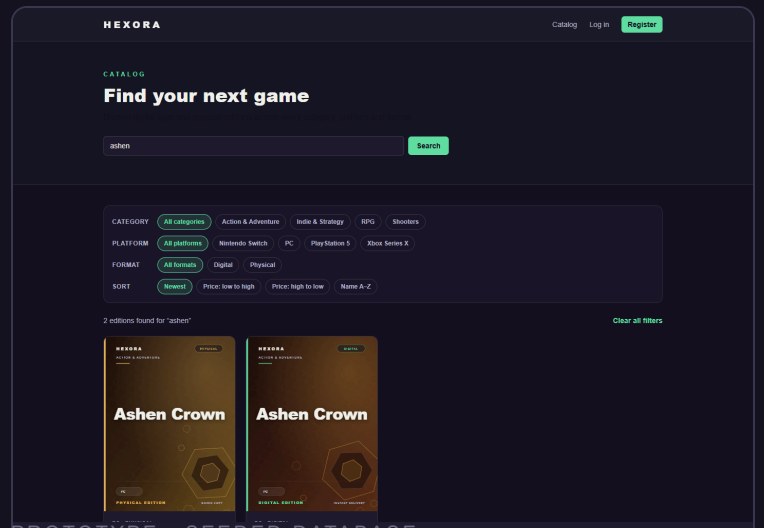
HOME · 1440 PX



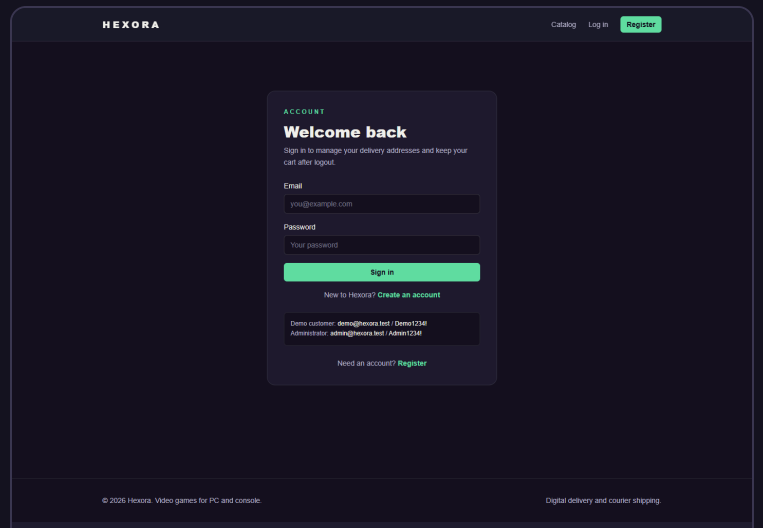
CATALOG · A1



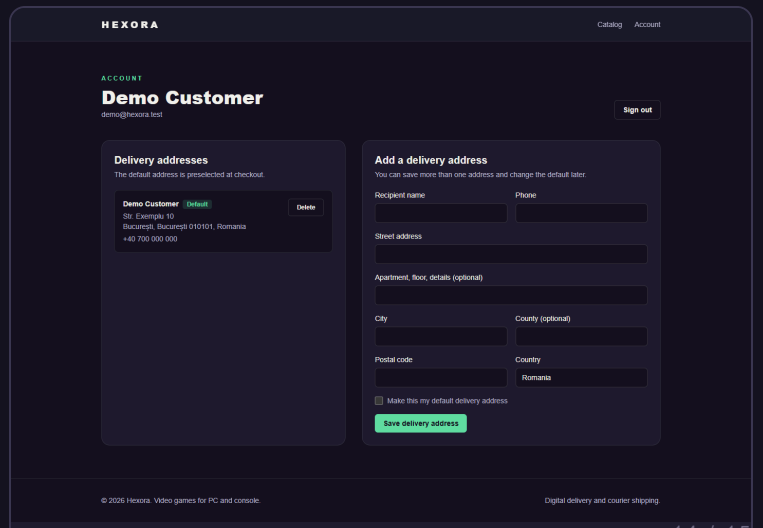
PRODUCT · SIBLINGS



SEARCH · A2



LOGIN · A4



ACCOUNT · ADDRESSES

PLAN, RISKS AND QUESTIONS

Weeks 8–12

WEEK	WORK	OWNER	STATUS
3	P1, repository, README	All	done
4	ERD, endpoints, wireframes	All	done
5	Seed data · A1 catalog · A4 accounts	Cosmin, Iulian	done
6	A2 search · Stripe setup · login prototype	Cosmin, Rares	done
7	P2 specification and live prototype	All	done
8	A3 cart · A5 checkout	Iulian, Rares	next
9	A6 confirmation · A7 stock · A8 history	Rares, Iulian	planned
10	A9 admin panel · A10 security	Cosmin, Rares	planned
11	A11 responsive · A12 tests · B4 + B5	Cosmin, Iulian	planned
12	P3 bug fixes, backup video, final README	All	planned

Risks we watch

- **Payments & stock:** webhook-first, conditional update, transaction tests.
- **Integration:** frozen contracts and small merges.
- **Scope:** required features first, feature freeze in week 11.

Thank you — questions

Repository, branch and PR links are in the README and the specification document.